

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE NAME: Fundamentals of Data Science

COURSE CODE: DSA04

COURSE OUTCOME COVERED

CO4: Develop machine learning models for classification, regression, and clustering, including performance evaluation and methodology comparison. (BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:100

**Annexure A
Industry Problem Solving**

Code of Conduct:

*I A.Sai Rohit (192472144) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: A.Sai Rohit

RUBRICS FOR CALCULATION

SCENARIO BASED ASSESSMENT							
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Level	Marks
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario		
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues		
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts		
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided		
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand		

Annexure A

Question 1: Real Estate Price Prediction Using Linear Regression

A real estate company wants to predict the selling price of houses based on house-related features.

House	Area sq.ft	Bedrooms	Age	Distance from City km	Price Lakhs
H1	900	2	12	10	45
H2	1100	2	10	8	55
H3	1300	3	8	6	68
H4	1500	3	5	5	82
H5	1800	4	3	3	115
H6	2000	4	2	2	135
H7	1000	2	15	12	40
H8	1600	3	6	4	90

Tasks

- Create the dataset using Python.
- Identify independent and dependent variables.
- Split the dataset into training and testing data.
- Implement Linear Regression.
- Predict house price for test data.
- Predict the price of a new house with:

Area	Bedrooms	Age	Distance
1700	3	4	4

- Evaluate the model using MAE, MSE, RMSE, and R^2 score.
- Interpret the result and explain whether the model is useful for real estate price prediction.

a) Create the Dataset Using Python

The dataset contains information about eight houses. The input features are Area, Bedrooms, Age, and Distance from City. The target variable is Price in lakhs.

b) Identify Independent and Dependent Variables

Independent variables (X):

- Area
- Bedrooms
- Age
- Distance from City

Dependent variable (y):

- Price

The independent variables are used to predict the dependent variable.

c) Split the Dataset into Training and Testing Data

The dataset is divided into training and testing data using an **80:20 ratio**. The training data is used to train the model, while the testing data is used to evaluate its performance.

d) Implement Linear Regression

Linear Regression is used because the target variable, house price, is a continuous numerical value. The model learns the relationship between house features and price.

e) Predict House Price for Test Data

The trained Linear Regression model is used to predict the prices of the test houses. The predicted prices are compared with their actual prices.

f) Predict the Price of a New House

New house details:

- Area = 1700 sq.ft
- Bedrooms = 3
- Age = 4 years
- Distance from City = 4 km

The model predicts the selling price of this house.

g) Model Evaluation

The model is evaluated using:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R^2 Score

These metrics measure the prediction error and goodness of fit of the model.

h) Interpretation

The Linear Regression model provides a strong prediction of house prices for the given dataset. A high R^2 value indicates that the selected house features explain a large portion of the variation in house prices.

The model can be useful for preliminary real-estate price prediction. However, the dataset contains only eight records, so a larger dataset would be required for reliable real-world predictions.

Code:

```
import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

data = {

    "Area": [900, 1100, 1300, 1500, 1800, 2000, 1000, 1600],

    "Bedrooms": [2, 2, 3, 3, 4, 4, 2, 3],

    "Age": [12, 10, 8, 5, 3, 2, 15, 6],

    "Distance": [10, 8, 6, 5, 3, 2, 12, 4],

    "Price": [45, 55, 68, 82, 115, 135, 40, 90]

}

df = pd.DataFrame(data)

print("DATASET")

print(df)

X = df[["Area", "Bedrooms", "Age", "Distance"]]

y = df["Price"]

X_train, X_test, y_train, y_test = train_test_split(

    X, y,

    test_size=0.2,

    random_state=42

)

model = LinearRegression()

model.fit(X_train, y_train)
```

```

y_pred = model.predict(X_test)

print("\nACTUAL VS PREDICTED PRICES")

for actual, predicted in zip(y_test, y_pred):

    print(

        "Actual Price:",

        actual,

        "Predicted Price:",

        round(predicted, 2)

    )

new_house = [[1700, 3, 4, 4]]

new_prediction = model.predict(new_house)

print("\nNEW HOUSE PREDICTION")

print("Area = 1700 sq.ft")

print("Bedrooms = 3")

print("Age = 4 years")

print("Distance = 4 km")

print(

    "Predicted House Price =",

    round(new_prediction[0], 2),

    "Lakhs"

)

mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)

```

```

print("\nMODEL EVALUATION")

print("MAE =", round(mae, 4))

print("MSE =", round(mse, 4))

print("RMSE =", round(rmse, 4))

print("R2 Score =", round(r2, 4))

print("\nMODEL COEFFICIENTS")

for feature, coefficient in zip(X.columns, model.coef_):

    print(feature, "=", round(coefficient, 4))

print("Intercept =", round(model.intercept_, 4))

```

Output:

```

... DATASET
   Area  Bedrooms  Age  Distance  Price
0   900         2   12         10    45
1  1100         2   10          8    55
2  1300         3    8          6    68
3  1500         3    5          5    82
4  1800         4    3          3   115
5  2000         4    2          2   135
6  1000         2   15         12    40
7  1600         3    6          4    90

ACTUAL VS PREDICTED PRICES
Actual Price: 55 Predicted Price: 51.19
Actual Price: 135 Predicted Price: 121.84

NEW HOUSE PREDICTION
Area = 1700 sq.ft
Bedrooms = 3
Age = 4 years
Distance = 4 km
Predicted House Price = 95.95 Lakhs

MODEL EVALUATION
MAE = 8.4887
MSE = 93.9204
RMSE = 9.6913
R2 Score = 0.9413

MODEL COEFFICIENTS
Area = 0.0458
Bedrooms = 10.4386
Age = -1.7108
Distance = 0.862
Intercept = -9.9036

```

Question 2: Customer Churn Prediction Using Logistic Regression

A telecom company wants to predict whether a customer will churn or not churn.

Customer	Monthly Bill	Complaints	Data Usage GB	Tenure Months	Churn
C1	500	0	25	36	0
C2	900	3	10	8	1
C3	750	2	15	12	1
C4	400	0	30	48	0
C5	1000	4	8	6	1
C6	550	1	22	30	0
C7	850	3	12	10	1
C8	450	0	28	40	0

Here,

Churn = 1 means customer leaves the company.

Churn = 0 means customer stays.

Tasks

- Create the dataset using Python.
- Identify the input features and target variable.
- Perform train-test split.
- Implement Logistic Regression.
- Predict churn status for test customers.
- Predict churn for a new customer:

Monthly Bill	Complaints	Data Usage GB	Tenure Months
800	2	14	10

g) Evaluate the model using accuracy, confusion matrix, precision, recall, and F1-score.

h) Explain why Logistic Regression is suitable for churn prediction.

a) Create the Dataset Using Python

The dataset contains customer information such as Monthly Bill, Complaints, Data Usage, Tenure Months and Churn.

b) Identify Input Features and Target Variable

Input features:

- Monthly Bill
- Complaints
- Data Usage
- Tenure Months

Target variable:

- Churn

Here:

Churn = 1 means the customer leaves the company.

Churn = 0 means the customer stays with the company.

c) Perform Train-Test Split

The dataset is divided into training and testing data using an **80:20 ratio**.

d) Implement Logistic Regression

Logistic Regression is used because customer churn is a **binary classification problem**. The output can have two classes: 0 or 1.

e) Predict Churn Status for Test Customers

The trained Logistic Regression model predicts whether each test customer will churn or stay.

f) Predict Churn for a New Customer

New customer:

- Monthly Bill = 800
- Complaints = 2
- Data Usage = 14 GB
- Tenure = 10 months

The trained model is used to predict the churn status.

g) Model Evaluation

The model is evaluated using:

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1-score

h) Why Logistic Regression is Suitable

Logistic Regression is suitable because churn prediction has two possible outcomes. It predicts the probability of a customer belonging to either the churn or non-churn class.

Therefore, Logistic Regression is appropriate for this binary classification problem.

Code:

```
import pandas as pd
```



```

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import
accuracy_score,confusion_matrix,precision_score,recall_score,f1_score

data = {

    "Monthly_Bill":[500,900,750,400,1000,550,850,450],

    "Complaints":[0,3,2,0,4,1,3,0],

    "Data_Usage":[25,10,15,30,8,22,12,28],

    "Tenure":[36,8,12,48,6,30,10,40],

    "Churn":[0,1,1,0,1,0,1,0]

}

df = pd.DataFrame(data)

print("DATASET")

print(df)

X = df[["Monthly_Bill","Complaints","Data_Usage","Tenure"]]

y = df["Churn"]

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)

model = LogisticRegression(max_iter=1000)

model.fit(X_train,y_train)

y_pred = model.predict(X_test)

print("\nACTUAL VS PREDICTED CHURN")

for actual,predicted in zip(y_test,y_pred):

    print("Actual:",actual,"Predicted:",predicted)


new_customer = [[800,2,14,10]]

new_prediction = model.predict(new_customer)

```

```
probability = model.predict_proba(new_customer)

print("\nNEW CUSTOMER PREDICTION")

print("Predicted Churn =",new_prediction[0])

print("Probability of Staying =",round(probability[0][0],4))

print("Probability of Churning =",round(probability[0][1],4))

accuracy = accuracy_score(y_test,y_pred)

cm = confusion_matrix(y_test,y_pred)

precision = precision_score(y_test,y_pred,zero_division=0)

recall = recall_score(y_test,y_pred,zero_division=0)

f1 = f1_score(y_test,y_pred,zero_division=0)


print("\nMODEL EVALUATION")

print("Accuracy =",round(accuracy,4))

print("Confusion Matrix:")

print(cm)

print("Precision =",round(precision,4))

print("Recall =",round(recall,4))

print("F1 Score =",round(f1,4))
```

Output:

```

*** DATASET
    Monthly_Bill  Complaints  Data_Usage  Tenure  Churn
0           500           0         25      36      0
1           900           3         10       8      1
2           750           2         15      12      1
3           400           0         30      48      0
4          1000           4          8       6      1
5           550           1         22      30      0
6           850           3         12      10      1
7           450           0         28      40      0

```

```

ACTUAL VS PREDICTED CHURN
Actual: 1 Predicted: 1
Actual: 0 Predicted: 0

```

```

NEW CUSTOMER PREDICTION
Predicted Churn = 1
Probability of Staying = 0.0
Probability of Churning = 1.0

```

```

MODEL EVALUATION
Accuracy = 1.0
Confusion Matrix:
[[1 0]
 [0 1]]
Precision = 1.0
Recall = 1.0
F1 Score = 1.0

```

Conclusion

The Logistic Regression model successfully predicts customer churn for the given test data. The new customer is predicted as **Churn = 1**, meaning the customer is likely to leave. Because the dataset contains only eight customers, the results should be treated as a demonstration rather than a production-level churn model.

Question 3: Marketing Sales Prediction Using Lasso Regression

A retail company wants to predict monthly sales based on advertisement spending across different platforms. The company wants to know which marketing channels are most important.

Month	TV Ads	Social Media Ads	Newspaper Ads	Email Campaigns	Sales Lakhs
M1	50	20	10	5	15
M2	60	25	12	6	18
M3	80	40	15	8	25
M4	100	60	20	10	32
M5	120	70	22	12	38
M6	40	15	8	4	12
M7	90	50	18	9	28

Month	TV Ads	Social Media Ads	Newspaper Ads	Email Campaigns	Sales Lakhs
M8	110	65	21	11	35

Tasks

- Create the dataset using Python.
- Identify features and target variable.
- Apply train-test split.
- Implement Lasso Regression.
- Use an alpha value such as 0.1 or 1.0.
- Predict sales for test data.
- Predict sales for a new campaign:

TV Ads	Social Media Ads	Newspaper Ads	Email Campaigns
95	55	18	9

- Evaluate using MAE, MSE, RMSE, and R^2 score.
- Display the coefficients and identify which features are reduced or selected by Lasso.
- Explain how Lasso helps in feature selection.

a) Create the Dataset Using Python

The dataset contains advertisement spending on TV, social media, newspapers and email campaigns, along with monthly sales.

b) Identify Features and Target Variable

Features:

- TV Ads
- Social Media Ads
- Newspaper Ads
- Email Campaigns

Target variable:

- Sales

c) Apply Train-Test Split

The dataset is divided into training and testing data using an **80:20 ratio**.

d) Implement Lasso Regression

Lasso Regression is used to predict sales and identify important marketing channels.

e) Alpha Value

The alpha value is set to:

$\alpha = 0.1$

Alpha controls the strength of L1 regularization.

f) Predict Sales for Test Data

The trained Lasso Regression model predicts the sales values for the test data.

g) Predict Sales for a New Campaign

New campaign:

- TV Ads = 95
- Social Media Ads = 55
- Newspaper Ads = 18
- Email Campaigns = 9

The model predicts the expected sales.

h) Model Evaluation

The model is evaluated using:

- MAE
- MSE
- RMSE
- R^2 Score

i) Display Coefficients

The Lasso coefficients are displayed for all marketing channels.

Features whose coefficients become zero are considered less important by the Lasso model.

j) How Lasso Helps in Feature Selection

Lasso Regression uses **L1 regularization**. It adds a penalty based on the absolute values of the coefficients.

This can force some coefficients to exactly zero. Therefore, Lasso can automatically remove less important features and perform feature selection.

Code:

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.model_selection import train_test_split
```

```

from sklearn.linear_model import Lasso

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

data = {

    "TV_Ads": [50, 60, 80, 100, 120, 40, 90, 110],

    "Social_Media_Ads": [20, 25, 40, 60, 70, 15, 50, 65],

    "Newspaper_Ads": [10, 12, 15, 20, 22, 8, 18, 21],

    "Email_Campaigns": [5, 6, 8, 10, 12, 4, 9, 11],

    "Sales": [15, 18, 25, 32, 38, 12, 28, 35]

}

df = pd.DataFrame(data)

print("DATASET")

print(df)

X = df[["TV_Ads", "Social_Media_Ads", "Newspaper_Ads", "Email_Campaigns"]]

y = df["Sales"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = Lasso(alpha=0.1)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\nACTUAL VS PREDICTED SALES")

for actual, predicted in zip(y_test, y_pred):

    print("Actual:", actual, "Predicted:", round(predicted, 2))

new_campaign = [[95, 55, 18, 9]]

new_sales = model.predict(new_campaign)

print("\nNEW CAMPAIGN PREDICTION")

print("Predicted Sales =", round(new_sales[0], 2), "Lakhs")

```

```

mae = mean_absolute_error(y_test,y_pred)

mse = mean_squared_error(y_test,y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test,y_pred)

print("\nMODEL EVALUATION")

print("MAE =",round(mae,4))

print("MSE =",round(mse,4))

print("RMSE =",round(rmse,4))

print("R2 Score =",round(r2,4))

print("\nLASSO COEFFICIENTS")

for feature,coefficient in zip(X.columns,model.coef_):

    print(feature,"=",round(coefficient,4))

print("Intercept =",round(model.intercept_,4))

print("\nFEATURE SELECTION")

for feature,coefficient in zip(X.columns,model.coef_):

    if coefficient == 0:

        print(feature,"-> Not Selected")

    else:

        print(feature,"-> Selected")

```

Output:

```

DATASET
TV_Ads    Social_Media_Ads    Newspaper_Ads    Email_Campaigns    Sales
...
0      50      20      10      5      15
1      60      25      12      6      18
2      80      40      15      8      25
3      100     60      20      10     32
4      120     70      22      12     38
5      40      15      8       4      12
6      90      50      18      9      28
7      110     65      21     11     35

ACTUAL VS PREDICTED SALES
Actual: 18 Predicted: 18.18
Actual: 12 Predicted: 11.91

NEW CAMPAIGN PREDICTION
Predicted Sales = 30.06 Lakhs

MODEL EVALUATION
MAE = 0.1338
MSE = 0.0198
RMSE = 0.1409
R2 Score = 0.9978

LASSO COEFFICIENTS
TV_Ads = 0.2768
Social_Media_Ads = 0.0732
Newspaper_Ads = 0.0
Email_Campaigns = 0.0
Intercept = -0.2589

FEATURE SELECTION
TV_Ads -> Selected
Social_Media_Ads -> Selected
Newspaper_Ads -> Not Selected
Email_Campaigns -> Not Selected

```

Conclusion

The Lasso model predicts the sales of the new campaign and also identifies the important marketing channels. Lasso is useful because its L1 regularization can reduce unimportant feature coefficients to zero, providing automatic feature selection.

Question 4: Electricity Consumption Prediction Using Ridge Regression

A smart energy company wants to predict household electricity consumption. Some features may be highly correlated, such as number of appliances and daily usage hours. Therefore, Ridge Regression is used.

House	Appliances	Usage Hours	House Size sq.ft	Occupants	Consumption Units
H1	5	4	800	3	120
H2	7	6	1000	4	190
H3	8	7	1200	4	240
H4	10	8	1500	5	320
H5	4	3	700	2	90
H6	6	5	900	3	160
H7	9	7	1300	5	280
H8	11	9	1600	6	350

Tasks

- Create the dataset using Python.
- Identify the independent and dependent variables.
- Split the data into training and testing sets.
- Implement Ridge Regression.
- Use an alpha value such as 1.0.

f) Predict electricity consumption for test data.

g) Predict consumption for a new house:

Appliances	Usage Hours	House Size	Occupants
8	6	1100	4

h) Evaluate using MAE, MSE, RMSE, and R² score.

i) Display the Ridge coefficients.

j) Explain how Ridge Regression reduces overfitting.

a) Create the Dataset Using Python

The dataset contains the number of appliances, usage hours, house size, occupants and electricity consumption. Ridge Regression is selected because some input variables may be correlated.

b) Identify Independent and Dependent Variables

Independent variables:

- Appliances
- Usage Hours
- House Size
- Occupants

Dependent variable:

- Electricity Consumption

c) Split the Data

The data is divided into training and testing sets using an **80:20 ratio**.

d) Implement Ridge Regression

Ridge Regression is implemented using `Ridge()` from Scikit-learn.

e) Alpha Value

The alpha value is:

$$\alpha = 1.0$$

f) Predict Electricity Consumption for Test Data

The trained Ridge model predicts electricity consumption for the test houses.

g) Predict Consumption for a New House

New house:

- Appliances = 8
- Usage Hours = 6
- House Size = 1100 sq.ft
- Occupants = 4

The trained model predicts the expected electricity consumption.

h) Model Evaluation

The model is evaluated using:

- MAE
- MSE
- RMSE
- R² Score

i) Display Ridge Coefficients

The coefficients of Appliances, Usage Hours, House Size and Occupants are displayed.

j) How Ridge Regression Reduces Overfitting

Ridge Regression uses **L2 regularization**. It adds a penalty based on the squared values of model coefficients.

This prevents coefficients from becoming excessively large and helps reduce overfitting. Ridge is particularly useful when independent variables are highly correlated because it helps make the model more stable.

Code:

```
import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.linear_model import Ridge

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

data = {

    "Appliances": [5, 7, 8, 10, 4, 6, 9, 11],

    "Usage_Hours": [4, 6, 7, 8, 3, 5, 7, 9],

    "House_Size": [800, 1000, 1200, 1500, 700, 900, 1300, 1600],
```

```

    "Occupants":[3,4,4,5,2,3,5,6],

    "Consumption":[120,190,240,320,90,160,280,350]

}

df = pd.DataFrame(data)

print("DATASET")

print(df)

X = df[["Appliances","Usage_Hours","House_Size","Occupants"]]

y = df["Consumption"]

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)

model = Ridge(alpha=1.0)

model.fit(X_train,y_train)

y_pred = model.predict(X_test)

print("\nACTUAL VS PREDICTED CONSUMPTION")

for actual,predicted in zip(y_test,y_pred):

    print("Actual:",actual,"Predicted:",round(predicted,2))

new_house = [[8,6,1100,4]]

new_consumption = model.predict(new_house)

print("\nNEW HOUSE PREDICTION")

print("Predicted Consumption =",round(new_consumption[0],2),"Units")

mae = mean_absolute_error(y_test,y_pred)

mse = mean_squared_error(y_test,y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test,y_pred)

print("\nMODEL EVALUATION")

print("MAE =",round(mae,4))

```

```

print("MSE =",round(mse,4))

print("RMSE =",round(rmse,4))

print("R2 Score =",round(r2,4))

print("\nRIDGE COEFFICIENTS")

for feature,coefficient in zip(X.columns,model.coef_):

    print(feature,"=",round(coefficient,4))

print("Intercept =",round(model.intercept_,4))

```

Output:

```

... DATASET
      Appliances  Usage_Hours  House_Size  Occupants  Consumption
0              5             4         800           3           120
1              7             6        1000           4           190
2              8             7        1200           4           240
3             10             8        1500           5           320
4              4             3         700           2            90
5              6             5         900           3           160
6              9             7        1300           5           280
7             11             9        1600           6           350

ACTUAL VS PREDICTED CONSUMPTION
Actual: 190 Predicted: 184.62
Actual: 160 Predicted: 152.45

NEW HOUSE PREDICTION
Predicted Consumption = 212.73 Units

MODEL EVALUATION
MAE = 6.4624
MSE = 42.9358
RMSE = 6.5525
R2 Score = 0.8092

RIDGE COEFFICIENTS
Appliances = 3.6817
Usage_Hours = 1.0696
House_Size = 0.2443
Occupants = 2.9836
Intercept = -103.8165

```

Conclusion

The Ridge Regression model predicts electricity consumption using household characteristics. Ridge regularization reduces the effect of large coefficients and helps handle multicollinearity between features. Therefore, it is suitable for this electricity-consumption prediction problem.

Question 5: Delivery Time Prediction Using Linear Regression

A logistics company wants to predict delivery time based on distance, traffic level, number of packages, and weather score.

Delivery	Distance km	Traffic Level	Packages	Weather Score	Time Hours
D1	10	2	20	8	1.2
D2	20	4	30	7	2.4
D3	30	6	40	6	3.5
D4	40	8	50	5	4.8
D5	50	9	60	4	6.0
D6	15	3	25	8	1.8
D7	35	7	45	5	4.2
D8	25	5	35	6	3.0

Tasks

- Create the dataset.
- Identify input and output variables.
- Split the data into train and test sets.
- Implement Linear Regression.
- Predict delivery time for test records.
- Predict delivery time for:

Distance	Traffic Level	Packages	Weather Score
28	6	38	6

- Evaluate using MAE, MSE, RMSE, and R^2 score.
- Explain how this model can help a logistics company plan delivery schedules.

- Create the Dataset

The dataset is created using Python with eight delivery records.

- Identify Input and Output Variables

Input variables:

- Distance
- Traffic Level
- Packages
- Weather Score

Output variable:

- Delivery Time

- Split the Data

The dataset is divided into training and testing data using an **80:20 ratio**.

d) Implement Linear Regression

Linear Regression is used to learn the relationship between delivery-related factors and delivery time.

e) Predict Delivery Time for Test Records

The trained model predicts delivery time for the test records.

f) Predict Delivery Time for a New Delivery

Given:

- Distance = 28 km
- Traffic Level = 6
- Packages = 38
- Weather Score = 6

Predicted delivery time $\approx$ 3.32 hours.

g) Model Evaluation

The model is evaluated using:

- MAE = **0.0452**
- MSE = **0.0023**
- RMSE = **0.0480**
- R^2 = **0.9744**

h) Interpretation

The model gives a high R^2 score of approximately 0.9744, indicating that it explains most of the variation in delivery time for this dataset. A logistics company can use such predictions to estimate delivery schedules, allocate resources and provide more accurate expected delivery times.

Code:

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
data={  
    "Distance":[10,20,30,40,50,15,35,25],  
    "Traffic":[2,4,6,8,9,3,7,5],  
    "Packages":[20,30,40,50,60,25,45,35],  
    "Weather":[8,7,6,5,4,8,5,6],  
    "Time":[1.2,2.4,3.5,4.8,6.0,1.8,4.2,3.0]  
}
```

```
df=pd.DataFrame(data)
```

```
print(df)
```

```
X=df[["Distance","Traffic","Packages","Weather"]]
```

```
y=df["Time"]
```

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
```

```
model=LinearRegression()
```

```
model.fit(X_train,y_train)
```

```
y_pred=model.predict(X_test)
```

```
print("\nTEST PREDICTIONS")
```

```
for actual,predicted in zip(y_test,y_pred):
```

```
    print("Actual:",actual,"Predicted:",round(predicted,2))
```

```

new_delivery=[[28,6,38,6]]

new_prediction=model.predict(new_delivery)

print("\nNEW DELIVERY")

print("Predicted Delivery Time =",round(new_prediction[0],2),"Hours")


mae=mean_absolute_error(y_test,y_pred)

mse=mean_squared_error(y_test,y_pred)

rmse=np.sqrt(mse)

r2=r2_score(y_test,y_pred)


print("\nMODEL EVALUATION")

print("MAE =",round(mae,4))

print("MSE =",round(mse,4))

print("RMSE =",round(rmse,4))

print("R2 Score =",round(r2,4))

```

Output:

```

    Distance  Traffic  Packages  Weather  Time
0         10         2         20         8     1.2
1         20         4         30         7     2.4
2         30         6         40         6     3.5
3         40         8         50         5     4.8
4         50         9         60         4     6.0
5         15         3         25         8     1.8
6         35         7         45         5     4.2
7         25         5         35         6     3.0

TEST PREDICTIONS
Actual:  2.4 Predicted:  2.37
Actual:  1.8 Predicted:  1.74

NEW DELIVERY
Predicted Delivery Time = 3.32 Hours

MODEL EVALUATION
MAE = 0.0452
MSE = 0.0023
RMSE = 0.048
R2 Score = 0.9744

```


Question 6: Loan Approval Prediction Using Logistic Regression

A bank wants to predict whether a loan application should be approved or rejected.

Applicant	Income	Credit Score	Existing Loan	Employment Years	Approved
A1	25000	580	200000	1	0
A2	30000	600	180000	2	0
A3	45000	680	120000	3	1
A4	60000	750	80000	5	1
A5	70000	800	50000	6	1
A6	28000	590	220000	1	0
A7	55000	720	100000	4	1
A8	35000	620	160000	2	0

Here,

Approved = 1 means loan approved.

Approved = 0 means loan rejected.

Tasks

- Create the dataset in Python.
- Identify independent and dependent variables.
- Split the dataset.
- Implement Logistic Regression.
- Predict approval status for test data.
- Predict loan approval for a new applicant:

Income	Credit Score	Existing Loan	Employment Years
50000	700	100000	4

g) Evaluate using accuracy, confusion matrix, precision, recall, and F1-score.

h) Explain why Logistic Regression is suitable for loan approval prediction.

a) Create the Dataset

The eight loan application records are created using Python.

b) Identify Independent and Dependent Variables

Independent variables:

- Applicant Income
- Credit Score
- Existing Loan
- Employment Years

Dependent variable:

- Approved

c) Split the Dataset

The dataset is divided into training and testing data using an **80:20 ratio**.

d) Implement Logistic Regression

Logistic Regression is implemented because loan approval has two possible outcomes: approved or rejected.

e) Predict Approval Status for Test Data

The trained model predicts whether the test loan applications are approved or rejected.

f) Predict Loan Approval for a New Applicant

Given:

- Income = 50,000
- Credit Score = 700
- Existing Loan = 100,000
- Employment Years = 4

Predicted Approval = 1, meaning the loan is predicted to be approved.

g) Model Evaluation

Using the given 80:20 split:

- Accuracy = **1.0000**
- Confusion Matrix = `[[2]]`

Because the two test records happen to both belong to the rejected class, precision, recall and F1-score for class 1 are not defined for this particular split. In the code, `zero_division=0` is used so they are reported as **0.0** rather than producing an error.

h) Why Logistic Regression is Suitable

Logistic Regression is suitable because loan approval is a **binary classification problem**. The model predicts one of two outcomes: approved (1) or rejected (0).

Code:

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import
accuracy_score,confusion_matrix,precision_score,recall_score,f1_score


data={

    "Income":[25000,30000,45000,60000,70000,28000,55000,35000],

    "Credit":[580,600,680,750,800,590,720,620],

    "Loan":[200000,180000,120000,80000,50000,220000,100000,160000],

    "Employment":[1,2,3,5,6,1,4,2],

    "Approved":[0,0,1,1,1,0,1,0]

}


df=pd.DataFrame(data)

print(df)


X=df[["Income","Credit","Loan","Employment"]]

y=df["Approved"]

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

model=LogisticRegression(max_iter=1000)

model.fit(X_train,y_train)

y_pred=model.predict(X_test)

print("\nTEST PREDICTIONS")

for actual,predicted in zip(y_test,y_pred):

    print("Actual:",actual,"Predicted:",predicted)

new_applicant=[[50000,700,100000,4]]

new_prediction=model.predict(new_applicant)

```

```
probability=model.predict_proba(new_applicant)

print("\nNEW APPLICANT")

print("Predicted Approval =",new_prediction[0])

print("Probability of Rejection =",round(probability[0][0],4))

print("Probability of Approval =",round(probability[0][1],4))

accuracy=accuracy_score(y_test,y_pred)

cm=confusion_matrix(y_test,y_pred)

precision=precision_score(y_test,y_pred,zero_division=0)

recall=recall_score(y_test,y_pred,zero_division=0)

f1=f1_score(y_test,y_pred,zero_division=0)

print("\nMODEL EVALUATION")

print("Accuracy =",round(accuracy,4))

print("Confusion Matrix:")

print(cm)

print("Precision =",round(precision,4))

print("Recall =",round(recall,4))

print("F1 Score =",round(f1,4))
```

Output:

```

Income  Credit  Loan  Employment  Approved
0  25000    580  200000         1         0
1  30000    600  180000         2         0
2  45000    680  120000         3         1
3  60000    750   80000         5         1
4  70000    800   50000         6         1
5  28000    590  220000         1         0
6  55000    720  100000         4         1
7  35000    620  160000         2         0

TEST PREDICTIONS
Actual: 0 Predicted: 0
Actual: 0 Predicted: 0

NEW APPLICANT
Predicted Approval = 1
Probability of Rejection = 0.0
Probability of Approval = 1.0

MODEL EVALUATION
Accuracy = 1.0
Confusion Matrix:
[[2]]
Precision = 0.0
Recall = 0.0
F1 Score = 0.0

```

Question 7: Healthcare Treatment Cost Prediction Using Lasso Regression

A hospital wants to predict treatment cost based on patient health parameters. Since many medical features are available, the hospital wants to use Lasso Regression to identify important predictors.

Patient	Age	BP	Sugar Level	BMI	Previous Visits	Treatment Cost
P1	25	115	90	22	1	5000
P2	35	125	110	25	2	8000
P3	45	140	150	29	3	15000
P4	55	155	180	32	5	25000
P5	60	165	200	35	6	32000
P6	30	120	100	24	1	7000
P7	50	150	170	31	4	22000
P8	40	135	130	27	2	12000

Tasks

- Create the dataset using Python.
- Identify features and target variable.
- Split into training and testing sets.
- Implement Lasso Regression.
- Try alpha values such as 0.1, 1.0, and 10.0.
- Predict treatment cost for test patients.
- Predict treatment cost for a new patient:

Age	BP	Sugar Level	BMI	Previous Visits
48	145	160	30	4

h) Evaluate using MAE, MSE, RMSE, and R^2 score.

i) Print the model coefficients.

j) Explain which medical features are important based on Lasso coefficients.

a) Create the Dataset

The eight patient records are created using Python.

b) Identify Features and Target Variable

Features:

- Age
- BP
- Sugar Level
- BMI
- Previous Visits

Target:

- Treatment Cost

c) Split into Training and Testing Sets

The dataset is divided into training and testing data using an **80:20 ratio**.

d) Implement Lasso Regression

Lasso Regression is implemented to predict treatment cost and identify important medical features.

e) Try Different Alpha Values

The model is tested with:

- $\alpha = 0.1$
- $\alpha = 1.0$
- $\alpha = 10.0$

f) Predict Treatment Cost for Test Patients

The trained models predict treatment costs for the test patients.

g) Predict Treatment Cost for a New Patient

Given:

- Age = 48
- BP = 145
- Sugar Level = 160
- BMI = 30
- Previous Visits = 4

For $\alpha = 0.1$, predicted treatment cost is approximately:

₹19,594.75

h) Evaluate the Model

For $\alpha = 0.1$:

- MAE = **1064.96**
- MSE = **1,934,326.15**
- RMSE = **1390.80**
- $R^2 = -6.7373$

The negative R^2 occurs because the test set is extremely small and the dataset contains only eight records. This indicates that the model should not be considered reliable for real-world prediction based on this tiny dataset.

Code:

```
import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.linear_model import Lasso

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

data={

    "Age":[25,35,45,55,60,30,50,40],

    "BP":[115,125,140,155,165,120,150,135],

    "Sugar":[90,110,150,180,200,100,170,130],

    "BMI":[22,25,29,32,35,24,31,27],

    "Visits":[1,2,3,5,6,1,4,2],

    "Cost":[5000,8000,15000,25000,32000,7000,22000,12000]

}
```

```
df=pd.DataFrame(data)

print(df)

X=df[["Age","BP","Sugar","BMI","Visits"]]

y=df["Cost"]

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

new_patient=[[48,145,160,30,4]]

for alpha in [0.1,1.0,10.0]:

    model=Lasso(alpha=alpha,max_iter=100000)

    model.fit(X_train,y_train)

    y_pred=model.predict(X_test)

    new_prediction=model.predict(new_patient)

    mae=mean_absolute_error(y_test,y_pred)

    mse=mean_squared_error(y_test,y_pred)

    rmse=np.sqrt(mse)

    r2=r2_score(y_test,y_pred)

    print("\nALPHA =",alpha)

    print("Test Predictions:")

    for actual,predicted in zip(y_test,y_pred):

        print("Actual:",actual,"Predicted:",round(predicted,2))

    print("New Patient Cost =",round(new_prediction[0],2))

    print("MAE =",round(mae,2))

    print("MSE =",round(mse,2))

    print("RMSE =",round(rmse,2))

    print("R2 Score =",round(r2,4))

    print("Coefficients:")
```


for feature,coefficient in zip(X.columns,model.coef_):

print(feature,"=",round(coefficient,4))

Output:

```

  Age  BP  Sugar  BMI  Visits  Cost
0   25  115    90   22      1   5000
1   35  125   110   25      2   8000
2   45  140   150   29      3  15000
3   55  155   180   32      5  25000
4   60  165   200   35      6  32000
5   30  120   100   24      1   7000
6   50  150   170   31      4  22000
7   40  135   130   27      2  12000

ALPHA = 0.1
Test Predictions:
Actual: 8000 Predicted: 8170.43
Actual: 7000 Predicted: 5040.51
New Patient Cost = 19594.75
MAE = 1064.96
MSE = 1934326.15
RMSE = 1390.8
R2 Score = -6.7373
Coefficients:
Age = -753.4188
BP = 816.3621
Sugar = -23.2589
BMI = -13.7054
Visits = 3061.4964

ALPHA = 1.0
Test Predictions:
Actual: 8000 Predicted: 8145.1
Actual: 7000 Predicted: 5027.91
New Patient Cost = 19588.88
MAE = 1058.6
MSE = 1955101.27
RMSE = 1398.25
R2 Score = -6.8204
Coefficients:
Age = -757.0283
BP = 819.1026
Sugar = -20.9973
```

```

BMI = -26.8783
Visits = 3043.6743

ALPHA = 10.0
Test Predictions:
Actual: 8000 Predicted: 7961.52
Actual: 7000 Predicted: 5002.51
New Patient Cost = 19580.77
MAE = 1017.99
MSE = 1995728.76
RMSE = 1412.7
R2 Score = -6.9829
Coefficients:
Age = -800.3629
BP = 803.267
Sugar = 2.3215
BMI = -0.0
Visits = 2921.2773
```

Question 8: Manufacturing Defect Strength Prediction Using Ridge Regression

A manufacturing company wants to predict the strength score of a product based on machine parameters. Since temperature, pressure, and machine speed may be correlated, Ridge Regression is used.

Product	Temperature	Pressure	Machine Speed	Material Quality	Strength Score
P1	70	30	100	8	75
P2	75	35	110	7	78
P3	80	38	120	6	80
P4	85	42	125	6	82
P5	90	45	130	5	79
P6	95	50	140	4	76
P7	78	36	115	7	81
P8	88	44	128	5	80

Tasks

- Create the dataset.
- Identify the independent variables and dependent variable.
- Split into training and testing data.
- Implement Ridge Regression.
- Use alpha values such as 0.1, 1.0, and 10.0.
- Predict strength score for test products.
- Predict strength score for a new product:

Temperature	Pressure	Machine Speed	Material Quality
82	40	122	6

- Evaluate using MAE, MSE, RMSE, and R^2 score.
- Display Ridge coefficients.
- Explain how Ridge handles multicollinearity in manufacturing data.

- Create the Dataset

The eight manufacturing product records are created using Python.

- Identify Independent and Dependent Variables

Independent variables:

- Temperature
- Pressure
- Machine Speed
- Material Quality

Dependent variable:

- Strength Score

c) Split into Training and Testing Data

The dataset is divided into training and testing sets using an **80:20 ratio**.

d) Implement Ridge Regression

Ridge Regression is used to predict the strength score of manufactured products.

e) Use Different Alpha Values

The model is tested using:

- $\alpha = 0.1$
- $\alpha = 1.0$
- $\alpha = 10.0$

f) Predict Strength Score for Test Products

The trained Ridge models predict strength scores for the test products.

g) Predict Strength Score for a New Product

Given:

- Temperature = 82
- Pressure = 40
- Machine Speed = 122
- Material Quality = 6

For $\alpha = 1.0$, the predicted strength score is approximately:

80.89

h) Evaluate the Model

For $\alpha = 1.0$:

- MAE = **3.3354**
- MSE = **21.5103**
- RMSE = **4.6379**
- $R^2 = -20.5103$

The negative R^2 is mainly due to the extremely small test set of only two products. Therefore, the model should not be considered reliable for real-world manufacturing prediction without more data.

Code:

```
import pandas as pd
```

```

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.linear_model import Ridge

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

data={

"Temperature":[70,75,80,85,90,95,78,88],

"Pressure":[30,35,38,42,45,50,36,44],

"Speed":[100,110,120,125,130,140,115,128],

"Quality":[8,7,6,6,5,4,7,5],

"Strength":[75,78,80,82,79,76,81,80]

}

df=pd.DataFrame(data)

print(df)


X=df[["Temperature","Pressure","Speed","Quality"]]

y=df["Strength"]


X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

new_product=[[82,40,122,6]]

for alpha in [0.1,1.0,10.0]:

    model=Ridge(alpha=alpha)

    model.fit(X_train,y_train)

    y_pred=model.predict(X_test)

    new_prediction=model.predict(new_product)

    mae=mean_absolute_error(y_test,y_pred)

```

```

mse=mean_squared_error(y_test,y_pred)

rmse=np.sqrt(mse)

r2=r2_score(y_test,y_pred)

print("\nALPHA =",alpha)

print("Test Predictions:")

for actual,predicted in zip(y_test,y_pred):

    print("Actual:",actual,"Predicted:",round(predicted,2))

print("New Product Strength =",round(new_prediction[0],2))

print("MAE =",round(mae,4))

print("MSE =",round(mse,4))

print("RMSE =",round(rmse,4))

print("R2 Score =",round(r2,4))

print("Coefficients:")

for feature,coefficient in zip(X.columns,model.coef_):

    print(feature,"=",round(coefficient,4))

```

Output:

```

Temperature  Pressure  Speed  Quality  Strength
0           70         30    100         8         75
1           75         35    110         7         78
2           80         38    120         6         80
3           85         42    125         6         82
4           90         45    130         5         79
5           95         50    140         4         76
6           78         36    115         7         81
7           88         44    128         5         80

ALPHA = 0.1
Test Predictions:
Actual: 78 Predicted: 78.34
Actual: 76 Predicted: 82.76
New Product Strength = 81.26
MAE = 3.5512
MSE = 22.8999
RMSE = 4.7854
R2 Score = -21.8999
Coefficients:
Temperature = -0.9816
Pressure = 0.4365
Speed = 0.8881
Quality = 3.0483

ALPHA = 1.0
Test Predictions:
Actual: 78 Predicted: 78.11
Actual: 76 Predicted: 82.56
New Product Strength = 80.89
MAE = 3.3354
MSE = 21.5103
RMSE = 4.6379

```

```

R2 Score = -20.5103
Coefficients:
Temperature = -0.6724
Pressure = -0.1134
Speed = 0.7587
Quality = 1.0556

ALPHA = 10.0
Test Predictions:
Actual: 78 Predicted: 78.11
Actual: 76 Predicted: 82.46
New Product Strength = 80.26
MAE = 3.2834
MSE = 20.8738
RMSE = 4.5688
R2 Score = -19.8738
Coefficients:
Temperature = -0.2663
Pressure = -0.1131
Speed = 0.3933
Quality = 0.1406

```

Question 9: Student Salary Package Prediction Using Linear Regression

A university placement cell wants to predict the expected salary package of students.

Student	CGPA	Aptitude Score	Coding Score	Communication Score	Package LPA
S1	6.5	60	55	65	3.2
S2	7.0	65	60	70	4.0
S3	7.5	70	68	72	5.0
S4	8.0	78	80	75	6.5
S5	8.5	85	88	80	8.0
S6	9.0	90	92	85	10.0
S7	6.8	62	58	68	3.8
S8	8.2	80	82	78	7.0

Tasks

- Create the dataset using Python.
- Identify input and output variables.
- Split the dataset.
- Implement Linear Regression.
- Predict package for test students.
- Predict package for a new student:

CGPA	Aptitude Score	Coding Score	Communication Score
8.1	82	85	78

- Evaluate using MAE, MSE, RMSE, and R^2 score.
- Interpret the result from the placement cell perspective.

a) Create the Dataset Using Python

The dataset is created using Python with eight student records containing academic and skill-related information.

b) Identify Input and Output Variables

Input variables:

- CGPA
- Aptitude Score
- Coding Score
- Communication Score

Output variable:

- Package in LPA

c) Split the Dataset

The dataset is divided into training and testing sets using an **80:20 ratio**.

d) Implement Linear Regression

Linear Regression is used to predict the expected salary package based on the student's academic and skill scores.

e) Predict Package for Test Students

The trained Linear Regression model predicts the salary packages for the test students.

f) Predict Package for a New Student

The new student's details are:

- CGPA = 8.1
- Aptitude Score = 82
- Coding Score = 85
- Communication Score = 78

Predicted Package = approximately 7.63 LPA.

g) Model Evaluation

The model is evaluated using:

- MAE = **0.6670**
- MSE = **0.5812**
- RMSE = **0.7623**
- R^2 = **0.9354**

h) Interpretation from the Placement Cell Perspective

The model achieves an R^2 score of approximately 0.9354, indicating a strong relationship between the student's academic/skill scores and expected salary package for this dataset.

The placement cell can use the model as an initial tool to estimate expected packages and identify students who may require additional skill development. However, a larger dataset would be required for reliable real-world salary prediction.

Code:

```
import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score


data={

"CGPA":[6.5,7.0,7.5,8.0,8.5,9.0,6.8,8.2],

"Aptitude":[60,65,70,78,85,90,62,80],

"Coding":[55,60,68,80,88,92,58,82],

"Communication":[65,70,72,75,80,85,68,78],

"Package":[3.2,4.0,5.0,6.5,8.0,10.0,3.8,7.0]

}


df=pd.DataFrame(data)

print("DATASET")

print(df)


X=df[["CGPA","Aptitude","Coding","Communication"]]

y=df["Package"]
```



```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
```

```
model=LinearRegression()
```

```
model.fit(X_train,y_train)
```

```
y_pred=model.predict(X_test)
```

```
print("\nTEST PREDICTIONS")
```

```
for actual,predicted in zip(y_test,y_pred):
```

```
    print("Actual:",actual,"Predicted:",round(predicted,2))
```

```
new_student=[[8.1,82,85,78]]
```

```
new_prediction=model.predict(new_student)
```

```
print("\nNEW STUDENT")
```

```
print("Predicted Package =",round(new_prediction[0],2),"LPA")
```

```
mae=mean_absolute_error(y_test,y_pred)
```

```
mse=mean_squared_error(y_test,y_pred)
```

```
rmse=np.sqrt(mse)
```

```
r2=r2_score(y_test,y_pred)
```

```
print("\nMODEL EVALUATION")
```

```
print("MAE =",round(mae,4))
```

```
print("MSE =",round(mse,4))

print("RMSE =",round(rmse,4))

print("R2 Score =",round(r2,4))
```

Output:

```
DATASET
  CGPA  Aptitude  Coding  Communication  Package
0   6.5       60     55           65       3.2
1   7.0       65     60           70       4.0
2   7.5       70     68           72       5.0
3   8.0       78     80           75       6.5
4   8.5       85     88           80       8.0
5   9.0       90     92           85      10.0
6   6.8       62     58           68       3.8
7   8.2       80     82           78       7.0

TEST PREDICTIONS
Actual: 4.0 Predicted: 4.3
Actual: 10.0 Predicted: 8.96

NEW STUDENT
Predicted Package = 7.63 LPA

MODEL EVALUATION
MAE = 0.667
MSE = 0.5812
RMSE = 0.7623
R2 Score = 0.9354
```

Question 10: Employee Attrition Prediction Using Logistic Regression

An HR department wants to predict whether an employee will leave the company or stay.

Employee	Experience	Satisfaction Score	Overtime Hours	Salary Increment %	Leave
E1	2	8	2	12	0
E2	3	7	4	10	0
E3	1	4	10	5	1
E4	5	6	6	8	0
E5	6	3	12	4	1
E6	8	2	14	3	1
E7	4	7	3	11	0
E8	7	4	11	5	1

Here,

Leave = 1 means employee may leave.

Leave = 0 means employee may stay.

Tasks

- a) Create the dataset using Python.
- b) Identify features and target variable.
- c) Split into training and testing data.
- d) Implement Logistic Regression.
- e) Predict attrition for test employees.
- f) Predict attrition for a new employee:

Experience	Satisfaction Score	Overtime Hours	Salary Increment %
5	4	9	6

- g) Evaluate using accuracy, confusion matrix, precision, recall, and F1-score.
- h) Explain how the HR team can use this prediction to reduce employee attrition.

a) Create the Dataset Using Python

The dataset is created using Python with eight employee records.

b) Identify Independent and Dependent Variables

Independent variables:

- Experience
- Satisfaction Score
- Overtime Hours
- Salary Increment %

Dependent variable:

- Leave

Here:

- **Leave = 1** → Employee may leave.
- **Leave = 0** → Employee may stay.

c) Split the Dataset

The dataset is divided into training and testing data using an **80:20 ratio**.

d) Implement Logistic Regression

Logistic Regression is used because employee attrition is a binary classification problem with two possible outcomes: leave or stay.

e) Predict Leave Status for Test Employees

The trained Logistic Regression model predicts whether the test employees are likely to leave or stay.

f) Predict Attrition for a New Employee

For the new employee:

- Experience = 5 years
- Satisfaction Score = 5
- Overtime Hours = 8
- Salary Increment = 6%

Predicted Leave = 1.

Therefore, the employee is predicted to be likely to leave.

The predicted probability of leaving is approximately **60.38%**.

g) Model Evaluation

The model is evaluated using:

- Accuracy = **1.0000**
- Confusion Matrix = **[[1, 0], [0, 1]]**
- Precision = **1.0000**
- Recall = **1.0000**
- F1-score = **1.0000**

h) Why Logistic Regression is Suitable

Logistic Regression is suitable because employee attrition is a **binary classification problem**. The model predicts whether an employee belongs to the leave class (1) or stay class (0).

The probability output can also help HR identify employees who have a higher risk of leaving.

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import
accuracy_score, confusion_matrix, precision_score, recall_score, f1_score

data={
"Experience":[2,3,1,5,6,8,4,7],
```

```

"Satisfaction":[8,7,4,6,3,2,7,4],
"Overtime":[2,4,10,6,12,14,3,11],
"Increment":[12,10,5,8,4,3,11,5],
"Leave":[0,0,1,0,1,1,0,1]
}

df=pd.DataFrame(data)
print("DATASET")
print(df)

X=df[["Experience","Satisfaction","Overtime","Increment"]]
y=df["Leave"]

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

model=LogisticRegression(max_iter=1000)
model.fit(X_train,y_train)

y_pred=model.predict(X_test)

print("\nTEST PREDICTIONS")
for actual,predicted in zip(y_test,y_pred):
    print("Actual:",actual,"Predicted:",predicted)

new_employee=[[5,5,8,6]]
new_prediction=model.predict(new_employee)
probability=model.predict_proba(new_employee)

print("\nNEW EMPLOYEE")
print("Predicted Leave =",new_prediction[0])
print("Probability of Staying =",round(probability[0][0],4))
print("Probability of Leaving =",round(probability[0][1],4))

accuracy=accuracy_score(y_test,y_pred)
cm=confusion_matrix(y_test,y_pred)
precision=precision_score(y_test,y_pred,zero_division=0)
recall=recall_score(y_test,y_pred,zero_division=0)
f1=f1_score(y_test,y_pred,zero_division=0)

print("\nMODEL EVALUATION")
print("Accuracy =",round(accuracy,4))
print("Confusion Matrix:")
print(cm)
print("Precision =",round(precision,4))
print("Recall =",round(recall,4))
print("F1 Score =",round(f1,4))

```

Output:

DATASET

	Experience	Satisfaction	Overtime	Increment	Leave
0	2	8	2	12	0
1	3	7	4	10	0
2	1	4	10	5	1
3	5	6	6	8	0
4	6	3	12	4	1
5	8	2	14	3	1
6	4	7	3	11	0
7	7	4	11	5	1

TEST PREDICTIONS

Actual: 0 Predicted: 0

Actual: 1 Predicted: 1

NEW EMPLOYEE

Predicted Leave = 1

Probability of Staying = 0.3962

Probability of Leaving = 0.6038

MODEL EVALUATION

Accuracy = 1.0

Confusion Matrix:

[[1 0]

[0 1]]

Precision = 1.0

Recall = 1.0

F1 Score = 1.0